ĐẠI HỌC QUỐC GIA TP HCM

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

—----------

ĐỒ ÁN MÔN HỌC XỬ LÝ NGÔN NGỮ TỰ NHIÊN

ĐỀ TÀI:

HỆ THỐNG HỎI ĐÁP TỰ ĐỘNG, TỰ ĐỘNG SINH MÃ NGUỒN VỚI NGÔN NGỮ TỰ NHIÊN

HUỲNH QUỐC BỮU - 240104026

TRẦN TÙNG - 250101074

GIẢNG VIÊN:
TS. NGUYỄN TRỌNG CHỈNH

TP HỒ CHÍ MINH – NĂM 2026

Mục lục

1 Giới thiệu vấn đề 2
2 Phân tích ngữ liệu để xác định đặc điểm và hướng giải quyết bài toán 3
3 Phương pháp giải quyết vấn đề được đặt ra trong đồ án. 4
3.1 Phân rã xác suất trong kiến trúc Retrieve-then-Generate 5
3.2 Module Retriever (Dense Retriever): 6
3.3 Module Generator (CodeT5 + FiD): 7
4 Triển khai thực nghiệm, đánh giá và so sánh 7
4.1 Thiết lập thực nghiệm 7
4.2 Kết quả quả thực nghiệm của tác giả (paper DocPrompting) 8
4.3 Kết quả thực chạy của hệ thống do nhóm cài đặt: 8
4.3.1 Thực ngiệm lại code của tác giả 9
4.3.2 Các api nhóm mở rộng thêm cho retriever and generator 10
5 Nhận xét kết quả nghiên cứu 13
5.1 Đánh giá mức độ đáp ứng mục tiêu 13
5.2 Tính mở rộng 13
5.3 Hạn chế & Hướng cải tiến: 13
6 Tài liệu tham khảo 13

1 Giới thiệu vấn đề

Trong bối cảnh các mô hình ngôn ngữ lớn (LLMs) ngày càng phát triển, bài toán tự động sinh mã nguồn từ ngôn ngữ tự nhiên (Natural Language to Code) đang thu hút sự chú ý lớn. Các mô hình hiện đại như T5, CodeT5, hay Codex đã đạt được kết quả ấn tượng. Tuy nhiên, một **hạn chế lớn** của các mô hình này là chúng học trực tiếp từ dữ liệu huấn luyện (ghi nhớ tĩnh), do đó chúng mặc định mọi thư viện hoặc hàm đều đã xuất hiện trong tập huấn luyện.

Trên thực tế, các hàm và thư viện lập trình (như Python) liên tục được cập nhật và thêm mới mỗi ngày. Hệ quả là các mô hình này không có khả năng tổng quát hóa hoặc **bị ảo giác (hallucaination)** khi gặp phải các mã nguồn chứa các hàm hoặc thư viện "chưa từng xuất hiện" (unseen functions/libraries).

Đối với các lập trình viên khi sử dùng các hàm và thư viện mới (unseen functions/libraries), họ thường tìm tới các nguồn tài liệu văn bản như code manuals để tìm hiểu chúng.

Để giải quyết vấn đề **unseen functions/libraries** này, Shuyan Zhou và cộng sự vào năm 2022 đã giới thiệu phương pháp sinh mã nguồn dựa vào tài liệu gọi là **DocPrompting [1].** Tác giả đề xuất

một hướng tiếp cận "mở" (open-book) được lấy cảm hứng từ lập trình viên con người: **truy hồi và đọc tài liệu hướng dẫn (documentation) trước khi viết code**.

The Actor	Encounters New API	The Action	The Result
The Developer	Reads the Manual	Writes precise syntax	Success
Standard LLMs	Relies on static training	Guesses syntax	Hallucination
DocPrompting	Retrieves **Docs** automatically	Generates informed syntax	Success

Hình 1-1 DocPrompting giải quyết vấn đề "ảo giác" của các LLMs

Trong đồ án này, nhóm bám sát phương pháp và mã nguồn của bài báo **DocPrompting [2]** để **xây dựng một hệ thống sinh code python** hoàn chỉnh. Hệ thống tự động truy hồi tài liệu từ kho dữ liệu thư viện Python, sau đó kết hợp với yêu cầu của người dùng để sinh ra mã Python hiệu quả, giải quyết vấn đề "unseen functions".

Mã nguồn của **DocPrompting** được tác giả thực nghiệm vào 2022 cũng khá cũ so với 2026 nên không tương thích với python và các thư viện mới chạy trên Google Colab hiện tại. Do đó, Nhóm cũng đã tiến hành sửa đổi version vài thư viện và bổ sung các rest api để hiện thực ý lại ý tưởng theo hướng trực quan hơn thay vì các console app của tác giả. Mã nguồn của Nhóm cũng được public trên github ở link reference **[3]**

2 Phân tích ngữ liệu để xác định đặc điểm và hướng giải quyết bài toán

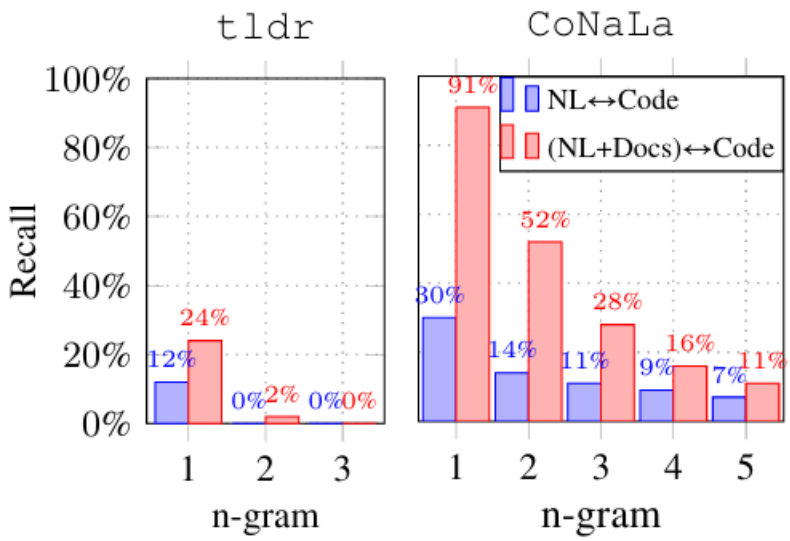
Đồ án tập trung vào miền ngôn ngữ Python, phân tích tập ngữ liệu CoNaLa **[4]** và kho tài liệu DevDocs **[5]**.

- **Tập ngữ liệu CoNaLa:**
  + **Nguồn gốc và phương pháp xây dựng:** CoNaLa (viết tắt của **Co**de and **Na**tural **La**nguage) là bộ dữ liệu điểm chuẩn (benchmark) nổi tiếng được nhóm nghiên cứu của P. Yin (2018) [4] xây dựng từ nền tảng Stack Overflow. Đặc thù của CoNaLa là nó chỉ tập trung khai thác các câu hỏi dạng *"how-to"* (làm thế nào để...) mang tính thực tiễn cao.
  + **Đặc điểm cấu trúc của dữ liệu:** Điểm sáng giá nhất của tập CoNaLa gốc là việc phân tách các câu trả lời trên Stack Overflow thành ba thành phần riêng biệt thay vì lấy toàn bộ:
    - *Intent (Ý định):* Mô tả bằng ngôn ngữ tự nhiên (thường là tiêu đề câu hỏi).
    - *Context (Ngữ cảnh):* Các đoạn mã cần thiết để chạy chương trình nhưng không trực tiếp giải quyết vấn đề (ví dụ: các câu lệnh import hoặc định nghĩa biến).

- *Snippet (Mã cốt lõi):* Phần mã thực sự giải quyết ý định của người dùng. Để tách được phần *Snippet* chuẩn xác khỏi *Context* gây nhiễu, dữ liệu CoNaLa được khai thác tự động thông qua một bộ phân loại kết hợp giữa các đặc trưng cấu trúc mã nguồn (Structural features) và đặc trưng tương quan (Correspondence features - sử dụng mô hình dịch máy thần kinh để tính xác suất khớp nhau giữa ngôn ngữ tự nhiên và code).

+ Đặc trưng quan trọng mà nhóm tác giả thiết lập là ở tập Test, **mỗi ví dụ đều chứa ít nhất một hàm Python chưa từng xuất hiện trong tập Train**. Điều này ép mô hình không được "học vẹt" mà phải thực sự hiểu tài liệu.

- **Kho tài liệu (Documentation Pool - $\boldsymbol{D}$):** Chứa 35.763 tài liệu từ DevDocs (bao gồm built-in, numpy, pandas, v.v.). Mỗi tài liệu mô tả một hàm.
- **Khoảng cách từ vựng:** Người dùng thường mô tả bằng ngôn ngữ tự nhiên ngắn gọn (VD: *"loại bỏ header khi ghi CSV"*), trong khi code yêu cầu cấu trúc chính xác (header=False). Phân tích từ bài báo gốc cho thấy việc bổ sung tài liệu hướng dẫn giúp **tăng tỷ lệ trùng lặp n-gram (n-gram overlap) từ 12% lên 24%**, tạo cầu nối trực tiếp nối liền "từ vựng ý định" và "từ vựng code".



Hình 2-1 Biểu đồ thể hiện sự gia tăng độ trùng lặp N-gram khi có tài liệu hướng dẫn

3 Phương pháp giải quyết vấn đề được đặt ra trong đồ án.

Từ câu hỏi ngôn ngữ tự nhiên n (NL intent n), mục đích sinh ra mã nguồn c viết bằng ngôn ngôn lập trình (PL) nào đó như Python.

S. Zhou trong paper DocPrompting đề xuất giải pháp Retrieve-then-Generate như sau:

Giả sử rằng mô hình truy cập tới tập tài liệu code $\boldsymbol{\mathcal{D}}$. Mỗi tài liệu $d_i \in \boldsymbol{\mathcal{D}}$ mô tả các thư viện, hàm và các tham số hữu dụng trong PL. Tập $\boldsymbol{\mathcal{D}}$ được xây dựng linh hoạt từ tập tài kiệu khả dụng mô tả thư viện và hàm trong PL hoặc trích xuất từ một dự án cụ thể.

Thay vì ánh xạ trực tiếp từ câu hỏi ngôn ngữ tự nhiên (n) sang mã nguồn (c), phương pháp sẽ định nghĩa quá trình sinh mã nguồn (c) có sự tham gia của tập tài liệu $\boldsymbol{\mathcal{D}}$. Bài toán được mô hình

hóa thành việc tối đa hóa xác suất sinh ra đoạn code c với điều kiện là và tập tài liệu $\boldsymbol{\mathcal{D}}$ $\boldsymbol{c} \coloneqq \boldsymbol{\arg max\, P(c \mid \mathcal{D}, n)}$.

3.1 Phân rã xác suất trong kiến trúc Retrieve-then-Generate

Mặc dù mô hình có thể sử dụng toàn bộ tập tài liệu $\boldsymbol{\mathcal{D}}$, nhưng chỉ có vài tài liệu có liên quan tới câu hỏi (intent) bất khả thi về mặt tính toán. Vì thế mô hình chỉ chọn ra tập con tài liệu $\boldsymbol{D_n} = \{d_1, d_2, .., d_k\} \subseteq \boldsymbol{\mathcal{D}}$ liên quan nhất tới n và dùng tập con này sinh ra mã nguồn c.

Bằng cách áp dụng luật xác suất toàn phần, phương pháp phân rã xác suất sinh mã nguồn c với điều kiện n $\boldsymbol{P(c \mid \mathcal{D}, n)}$ thành xác suất sinh tập tài liệu D_nvới điều kiện $\mathcal{D}$ $\boldsymbol{P(D_n \mid \mathcal{D}, n)}$ và xác suất sinh mã nguồn c với điều kiện D_n $\boldsymbol{P(c \mid D_n, n)}$. Sau đó biên hóa (marginalization) trên tập con tài liệu $\boldsymbol{D_n} \subseteq \boldsymbol{\mathcal{D}}$.

$$P(c \mid \mathcal{D}, n) = \sum_{D_n \subseteq \mathcal{D}} P(D_n \mid \mathcal{D}, n).P(c \mid D_n, n) \quad (\mathbf{3.1.1})$$

Tuy nhiên, việc tính toán trên toàn bộ không gian tập con là bất khả thi. Do đó, phương pháp sử dụng phép **xấp xỉ (approximation)** bằng cách chỉ lấy tập con chứa tài liệu có khả năng xảy ra cao nhất, ký hiệu là $\widehat{D_n}$:

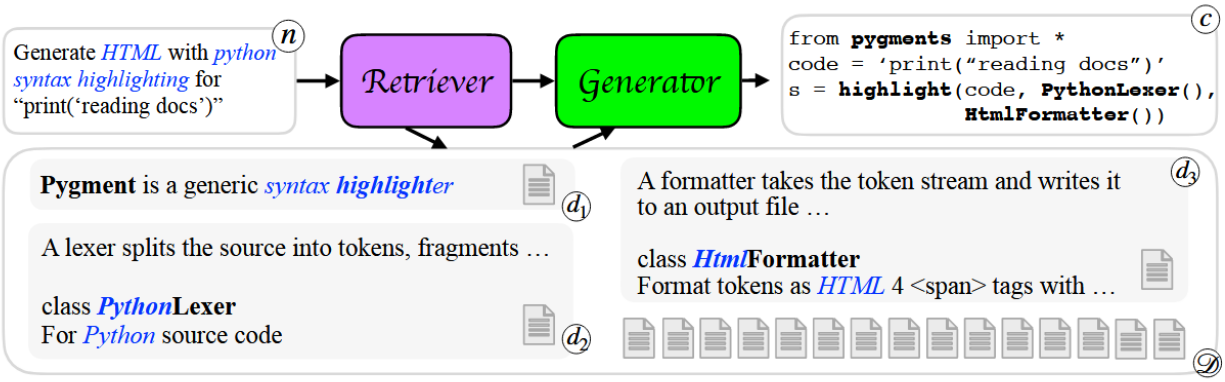
$$\widehat{D_n} \coloneqq \arg \max_{D_n \subseteq \mathcal{D}} P(D_n \mid \mathcal{D}, n) \quad (\mathbf{3.1.2})$$

Khi đó công thức (3.1.1) trở thành:

$$P(c \mid \mathcal{D}, n) \approx P\left(c \mid \widehat{D_n}, n\right) \cdot P\left(\widehat{D_n} \mid \mathcal{D}, n\right) = R.G \quad (\mathbf{3.1.3})$$

Công thức (3.1.3) chỉ ra rằng hệ thống sinh code có thể chia ra thành 2 module: **module Retriever** $R = P\left(\widehat{D_n} \mid \mathcal{D}, n\right)$ làm nhiệm vụ trích xuất các tài liệu tốt nhất, và **module Generator** $G = P\left(c \mid \widehat{D_n}, n\right)$ làm nhiệm vụ sinh code dựa trên các tài liệu đã trích xuất.

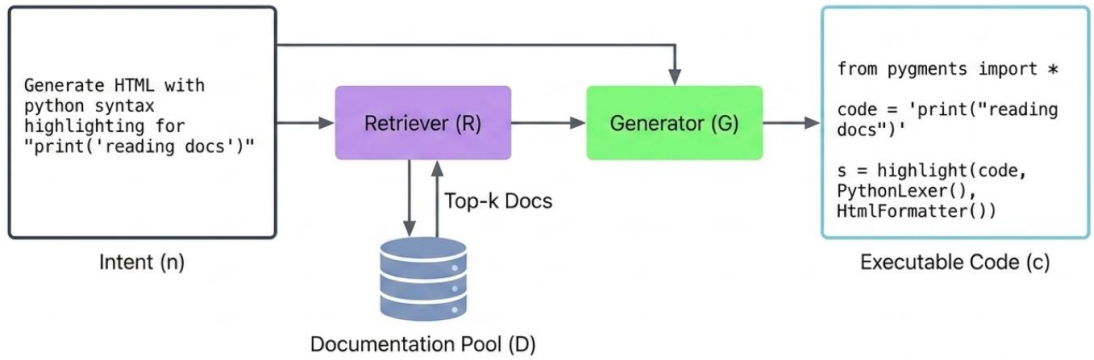
Cụ thể R tính toán độ tương đồng $\boldsymbol{s(d_i, n)}$ giữa n và các tài liệu $d_i \in \mathcal{D}$. Do đó, tập con $\boldsymbol{\widehat{D_n} \subseteq \mathcal{D}}$ **là top-k tài liệu với độ tương đồng cao nhất** $\boldsymbol{\widehat{D_n} = top_k_{d_i \in \mathcal{D}}\,(s(d_i, n))}$



Hình 3-1-1 Cách tiếp cận Retrieve-then-Generate

Tổng quan của cách tiếp cận này được thể hiện ở hình 3-1-1. Với câu hỏi n là *"Generate HTML with python syntax highlighting for "print('reading docs')"*, retriever R trích xuất ra ba tài liệu $\boldsymbol{d_1}$: *describes the syntax highlighting library* ***pygments***, $\boldsymbol{d_2}$: *describes the class* ***PythonLexer***, và

$\boldsymbol{d_3}$: *describes the* ***HtmlFormatter*** *class*. Sử dụng 3 tài liệu này kết hợp với câu hỏi n module generator G sinh ra code c sử dụng ***PythonLexer*** and ***HtmlFormatter*** từ thư viện ***pygment***.



Hình 3-1-2 Sơ đồ kiến trúc gen code với Retriever and Generator

3.2 Module Retriever (Dense Retriever):

S. Zhou trong paper DocPrompting đã thực nghiệm hai loại retriever chính là sparse retrievers and dense retrievers.

Với Sparse Retriever, tác giả truy hồi (retrieve) các tài liệu đại diện sử dụng các đặc trưng thưa thớt (sparse features) dựa vào tần suất từ như BM25 and TF-IDF.

Với **Dense Retriever**, tác giả dùng nghiên cứu trước đó của **Gao et al., 2021 [6]** để xây dựng. Diễn giải bên dưới mô tả cách xây dựng Dense Retriever với **SimCSE** model.

Cho triplet (n, c, D_n^*) với D_n^* là các tài liệu chỉ đường (oracle docs) cho n. Với mỗi $d_i^+ \in D_n^*$ đại diện cho cặp dương tính (n, d_i^+) và $d_j^- \notin D_n^*$ đại diện cho cặp âm tính (n, d_j^-). Tác giả đã huấn luyện Dense Retriever áp dụng kỹ thuật đối nghịch (contrastive fashion) bằng cách tìm ra độ tương đồng tối đa của cặp dương tính (n, d_i^+) và độ tương đồng tối thiểu của cặp âm tính (n, d_j^-).

Hàm mất mát của Dense Retriever $\mathcal{L}^r$ được định nghĩa như sau:

$$\mathcal{L}^r = -log \frac{e^{sim\left(h_n, h_{d_i^+}\right)}}{e^{sim\left(h_n, h_{d_i^+}\right)} + \sum_{d_j^- \in \mathcal{B} \setminus \mathcal{D}_n^*} e^{sim\left(h_n, h_{d_j^-}\right)}} \quad (3.2.1)$$

Với: h_x là vector embedding của tài liệu x qua bộ neural encoder,

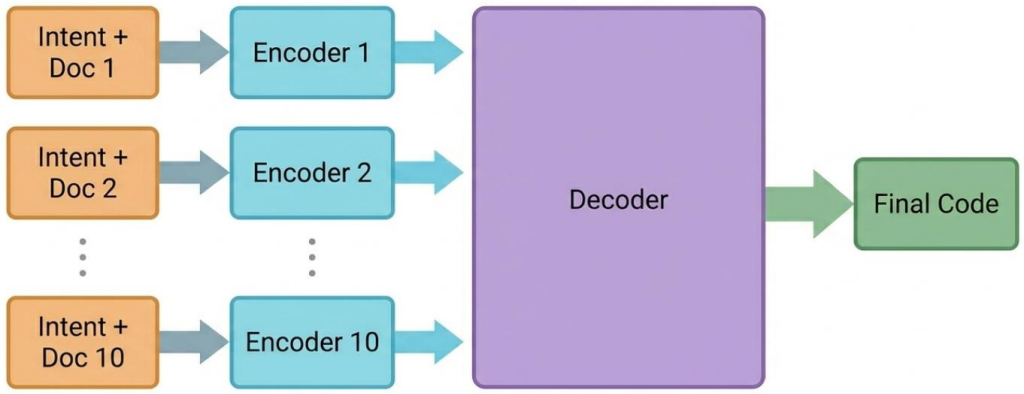
$\mathcal{B}$ là training batch,

$sim(h_x, h_y)$ là độ tương đồng của cosine (cosine similarity) của 2 vector h_x và h_y

Việc tối thiểu hóa hàm mất mát giúp mô hình học được không gian ngữ nghĩa chuẩn xác, từ đó làm cơ sở để trích xuất ra top-k tài liệu phù hợp nhất với yêu cầu.

3.3 Module Generator (CodeT5 + FiD):

Việc nối nối tiếp tất cả $\widehat{D_n}$ vào một chuỗi (prompt) sẽ vượt quá giới hạn đầu vào của CodeT5 (512 tokens). Do đó, tác giả giải quyết bằng kỹ thuật **Fusion-in-Decoder (FiD) [7]**. Kỹ thuật này mã hóa độc lập từng cặp (d_i, n) song song ở tầng Encoder, sau đó mới tổng hợp (attention) tất cả ở Decoder.



Hình 3-3-1 Generator với FiD

Tác giả finetune CodeT5 để sinh code c từ n và $\widehat{D_n}$ bằng cách tìm max hàm log-likelihood hoặc min của hàm mất mát $arg\,max\,log\,P(c \mid n, \widehat{D_n})$

c là **một chuỗi token code** $c = (c_1, c_2, .., c_T)$

Generator sinh code theo từng token, nên:

$$P\left(c \mid n, \widehat{D_n}\right) = \prod_{t=1}^{T} P\left(c_t \mid c_{<t}, n, \widehat{D_n}\right)$$

Do đó:

$$code\ c = arg\,max\,log\,P\left(c \mid n, \widehat{D_n}\right) = arg\,max \sum_{t=1}^{T} log\,P\left(c_t \mid c_{<t}, n, \widehat{D_n}\right)$$

Với:

n : intent/query

$\widehat{D}_n$: documents được retriever lấy ra

c_t: token code thứ t

$c_{<t}$: các token code trước đó

4 Triển khai thực nghiệm, đánh giá và so sánh

4.1 Thiết lập thực nghiệm

- **Phần cứng & Môi trường:** Google Colab (GPU T4), Python 3.x, Transformers 4.x, torch 2.x.

- **Baseline so sánh:** So sánh mô hình CodeT5 gốc (không có tài liệu) và CodeT5 + DocPrompting (có top 10 tài liệu)

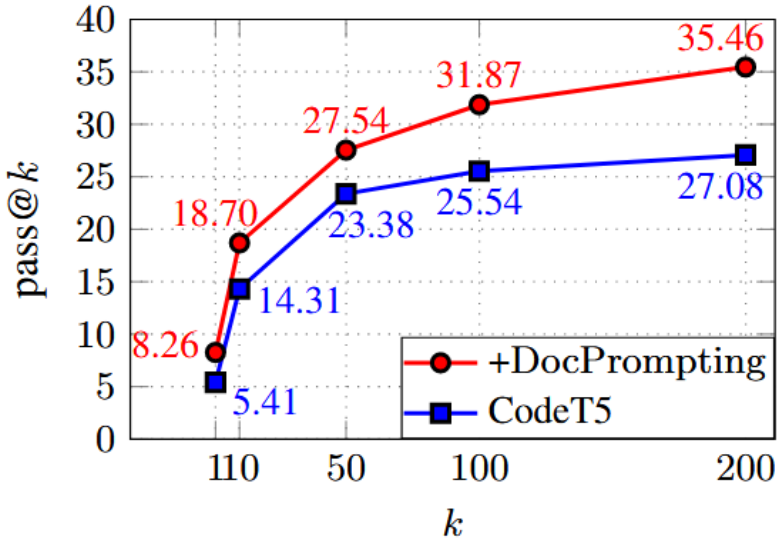
4.2 Kết quả quả thực nghiệm của tác giả (paper DocPrompting)

- Trên tập CoNaLa, DocPrompting cải thiện điểm BLEU từ 34.57 lên 36.22.
- Đặc biệt: Khả năng dự đoán đúng các hàm "chưa từng học" (Unseen function recall) tăng vọt từ 9.03 lên 18.30.

Model		BLEU	Recall	Recall$_{unseen}$
Codex *3-shots*	-	43.16	39.52	-
	+ DocPrompting	**43.47**	**39.87**	-
	+ DocPrompting oracle docs	50.59	57.84	-
T5	-	28.07	14.36	2.57
	+ DocPrompting	**30.04**	**21.34**	**8.24**
CodeT5	-	34.57	24.24	9.03
	+ DocPrompting	**36.22**	**27.80**	**18.30**
	+ DocPrompting oracle docs	49.04	72.20	63.91

Hình 4-2-1 DocPrompting cải thiện điểm BLEU và Unseen function recall

- Đánh giá thực thi, tỷ lệ sinh code chạy thành công (Pass@1) tăng thêm 2.85% (tương đương mức cải thiện tương đối 52%).



Hình 4-2-2 Kết quả thực thi Pass@k cho thấy CodeT5+DocPrompting (đường cam) vượt trội

4.3 Kết quả thực chạy của hệ thống do nhóm cài đặt:

Nhóm dự trên source code của tác giả cung cấp [2] để tiến hành chạy thực nghiệm.

Vì code của tác giả từ năm 2022 không tương thích trên môi trường Colab của Google hiện tại 2026 với các version python và thư viện mới. Nên nhóm đã tiến hành fork code về và chỉnh sửa vài thư viện cho tương thích [3]

4.3.1 Thực ngiệm lại code của tác giả

- **Kết quả chạy Dense Retriever**

```
# Tìm kiếm Top 10 tài liệu mô tả API Python liên quan nhất cho từng câu hỏi
!PYTHONPATH=/content/team19-docprompting python retriever/simcse/run_inference.py \
  --model_name "neulab/docprompting-codet5-python-doc-retriever" \
  --source_file data/conala/conala_nl.txt \
  --target_file data/conala/python_manual_firstpara.tok.txt \
  --source_embed_save_file data/conala/.tmp/src_embedding \
  --target_embed_save_file data/conala/.tmp/tgt_embedding \
  --sim_func cls_distance.cosine \
  --num_layers 12 \
  --save_file data/conala/retrieval_results.json
```

```
done embedding: (30755, 768)
source: data/conala/.tmp/src_embedding, target: data/conala/.tmp/tgt_embedding
(2879, 768) (30755, 768)
0.170   0.339   0.416   0.500   0.522   0.573   0.615   0.656   0.727   0.805   0.896   0.932
0.254   0.184   0.138   0.105   0.089   0.083   0.071   0.058   0.044   0.029   0.016   0.009
0.204   0.239   0.208   0.174   0.152   0.144   0.128   0.107   0.083   0.056   0.032   0.017
```

- **FiD Code Generator**

```
!PYTHONPATH=/content/team19-docprompting python generator/fid/test_reader_simple.py \
    --model_path models/generator/conala.fid.codet5.top10/checkpoint/best_dev/best_dev \
    --tokenizer_name models/generator/codet5-base \
    --eval_data data/conala/fid.cmd_test.codet5.t10.json \
    --per_gpu_batch_size 2 \
    --n_context 10 \
    --name conala.fid.codet5.top10 \
    --checkpoint_dir models/generator \
    --result_tag test_same \
    --main_port 81692
```

```
100% 272/272 [10:29<00:00,  2.31s/it]
src length: 543, tgt length: 543
[04/19/2026 08:48:54] {test_reader_simple.py:171} INFO - Total number of example 543
[04/19/2026 08:48:54] {test_reader_simple.py:172} INFO - {
  "bleu": 36.15
}
```

- **Xem kết quả:**

```python
eval_data_path = '/content/team19-docprompting/data/conala/fid.cmd_test.codet5.t10.json'
id_to_question = {}
if os.path.exists(eval_data_path):
    with open(eval_data_path, 'r', encoding='utf-8') as f:
        eval_data = json.load(f)
        id_to_question = {item['id']: item['question'] for item in eval_data}

result_path = "/content/team19-docprompting/models/generator/conala.fid.codet5.top10/test_results_test_same.json"
if os.path.exists(result_path):
    data = []
    with open(result_path, 'r', encoding='utf-8') as f:
        for line in f:
            if line.strip():
                data.append(json.loads(line))

    print("Hiển thị 3 kết quả Code đầu tiên:\n")
    for item in data[:3]:
        q_id = item.get('question_id')
        print("--- Ý định NL (Intent):", id_to_question.get(q_id, "Không rõ"))
        print("--- Code Đáp án (Gold):", item.get('gold'))
        print("+++ Code AI (Generated):", item.get('clean_code'))
        print("-" * 60)
else:
    print("File kết quả chưa được tạo ra.")
```

```
Hiển thị 3 kết quả Code đầu tiên:

--- Ý định NL (Intent): divide the values with same keys of two dictionary `d1` and `d2`
--- Code Đáp án (Gold): {k: (float(d2[k]) / d1[k]) for k in d2}
+++ Code AI (Generated): {k: d1[k] / d2[k] for k, v in list(d1.items())}
------------------------------------------------------------
--- Ý định NL (Intent): divide values associated with each key in dictionary `d1` from values associated with the same key in dictionary `d2`
--- Code Đáp án (Gold): dict((k, float(d2[k]) / d1[k]) for k in d2)
+++ Code AI (Generated): {d1[k] / d2[k] for k, v in list(d1.items())}
------------------------------------------------------------
--- Ý định NL (Intent): download "http://randomsite.com/file.gz" from http and save as "file.gz"
--- Code Đáp án (Gold): testfile = urllib.request.URLopener() testfile.retrieve('http://randomsite.com/file.gz', 'file.gz')
+++ Code AI (Generated): urllib.request.urlretrieve('http://randomsite.com/file.gz', 'http://randomsite.com/file.gz')
------------------------------------------------------------
```

4.3.2 Các api nhóm mở rộng thêm cho retriever and generator

```python
# ——— Start API + Swagger UI trên Colab ———
import os, subprocess, time

# Start server ở background
proc = subprocess.Popen(
    ["python", "team19_generator_pipeline_api.py", "--port", "8000"],
    env={**os.environ, "PYTHONPATH": "/content/team19-docprompting"},
    cwd="/content/team19-docprompting",
)

# Đợi model load xong (khoảng 30-60s)
print("⏳ Đang load models, đợi khoảng 60s...")
time.sleep(60)

# Tạo tunnel với ngrok
from pyngrok import ngrok
public_url = ngrok.connect(8000)
print(f"\n✅ Swagger UI: {public_url}/docs")
print(f"✅ API Base:   {public_url}")
```

```
⏳ Đang load models, đợi khoảng 60s...

✅ Swagger UI: NgrokTunnel: "https://unsaid-snort-uplifting.ngrok-free.dev" -> "http://localhost:8000"/docs
✅ API Base:   NgrokTunnel: "https://unsaid-snort-uplifting.ngrok-free.dev" -> "http://localhost:8000"
```

- **Retriever**

POST `/retrieve-queries` Retrieve Queries

Dense Retrieval: Tìm top-k tài liệu API Python liên quan nhất cho mỗi câu hỏi.

- Input: danh sách câu hỏi NL
- Output: top-k docs với doc_id, text, score

Request body required

Example Value | Schema

```
{
  "queries": [
    "sort a list of dictionaries by a value of the dictionary in python",
    "how to download a file from http url in python"
  ],
  "top_k": 10
}
```

Response body

```
{
  "results": [
    {
      "query": "sort a list of dictionaries by a value of the dictionary in python",
      "retrieved_docs": [
        {
          "rank": 1,
          "doc_id": "python.library.operator#operator.itemgetter",
          "text": "python operator itemgetter: Return a callable object that fetches item from its operand using the operand's __ge
titem__() method. If multiple items are specified, returns a tuple of lookup values. For example:",
          "score": 13.003414154052734
        },
        {
          "rank": 2,
          "doc_id": "python.library.functions#sorted",
          "text": "python sorted: Return a new sorted list from the items in iterable.",
          "score": 9.746530532836914
        },
        {
          "rank": 3,
          "doc_id": "python.library.stdtypes#list.sort",
          "text": "python list sort: This method sorts the list in place, using only < comparisons between items. Exceptions are no
t suppressed - if any comparison operations fail, the entire sort operation will fail (and the list will likely be left in a partia
lly modified state).",
          "score": 8.87022590637207
```

- **Generator**

POST `/generate-codes` Generate Codes

FiD Code Generation: Sinh code từ doc prompts đã chuẩn bị sẵn (FiD format).

- Input: danh sách {question, ctxs: [{title, text, score}]}
- Output: generated code cho mỗi câu hỏi

Request body required

Example Value | Schema

```
{
  "doc_prompts": [
    {
      "ctxs": [
        {
          "score": 0.85,
          "text": "sorted(iterable, *, key=None, reverse=False) Return a new sorted list.",
          "title": "sorted"
        }
      ],
      "question": "sort a list of dictionaries by a value of the dictionary"
    }
  ],
  "n_context": 10
}
```

Response body

```
{
  "results": [
    {
      "question": "sort a list of dictionaries by a value of the dictionary",
      "generated_code": "sorted(list, key=operator.itemgetter(1))"
    }
  ],
  "total_time_seconds": 0.978
}
```

- **Pipeline**

POST `/generate-pipeline-codes` Generate Pipeline Codes

Full Pipeline: NL Query → Dense Retriever → FiD Generator → Code.

- Input: chỉ cần danh sách câu hỏi NL
- Output: code sinh ra + top docs đã retrieve + timing

Request body required

Example Value | Schema

```
{
  "n_context": 10,
  "queries": [
    "sort a list of dictionaries by a value of the dictionary in python",
    "how to download a file from http url in python",
    "convert a string to datetime in python"
  ],
  "top_k": 10
}
```

Response body

```
{
  "results": [
    {
      "query": "sort a list of dictionaries by a value of the dictionary in python",
      "retrieved_docs": [
        {
          "doc_id": "python.library.operator#operator.itemgetter",
          "text": "python operator itemgetter: Return a callable object that fetches item from its operand using the operand's __getitem__() metho
tuple of lookup values. For",
          "score": 13.003414154052734
        },
        {
          "doc_id": "python.library.functions#sorted",
          "text": "python sorted: Return a new sorted list from the items in iterable.",
          "score": 9.746530532836914
        },
        {
          "doc_id": "python.library.stdtypes#list.sort",
          "text": "python list sort: This method sorts the list in place, using only < comparisons between items. Exceptions are not suppressed -
sort operation will fail (a",
          "score": 8.87022590637207
        },
```

Response body

```
        {
          "doc_id": "python.library.os#os.read",
          "text": "python os read: Read at most n bytes from file descriptor fd.",
          "score": 10.898857116699219
        }
      ],
      "generated_code": "urllib.request.urlretrieve(url, stream=True)",
      "retrieval_time": 0.011917988459269205,
      "generation_time": 1.2233535448710124
    },
```


5 Nhận xét kết quả nghiên cứu

5.1 Đánh giá mức độ đáp ứng mục tiêu

Phương pháp đáp ứng tiêu chí đánh giá "giải thích được". Phân rã xác suất (Công thức 3.1.1, 3.1.2) chứng minh được tính đúng đắn khi chia nhỏ bài toán. Việc Retriever cung cấp chữ ký hàm đã loại bỏ phần lớn hiện tượng hallucination của LLM

5.2 Tính mở rộng

Kho tài liệu $\mathcal{D}$ đóng vai trò như bộ nhớ ngoài. Thay vì mất hàng tuần huấn luyện lại LLM, ta chỉ cần cập nhật cơ sở dữ liệu hướng dẫn mới nhất.

5.3 Hạn chế & Hướng cải tiến:

Một vấn đề tồn đọng là khi các hàm có độ tương đồng ngữ nghĩa cực cao (VD: read_csv và to_csv), không gian nhúng (embedding space) theo Công thức 3.1.3 của Retriever có thể đánh giá sai khoảng cách $sim(h_x, h_y)$, dẫn đến trả về sai tài liệu. Giải pháp đề xuất trong tương lai là tích hợp một bộ lọc hậu xử lý (Post-filtering) so khớp tham số của mã sinh ra với chữ ký hàm trong tài liệu để chặn các luồng tham số sai lệch.

6 Tài liệu tham khảo

[1] S. Zhou, U. Alon, F. F. Xu, Z. Jiang, and G. Neubig, "Docprompting: Generating code by retrieving the docs," in *The Eleventh International Conference on Learning Representations*, 2022.

[2] S. Zhou, *Source code for DocPrompting: Generating Code by Retrieving the Docs*. (2022). Python. Accessed: Oct. 04, 2026. [Online]. Available: https://github.com/shuyanzhou/docprompting
[3] H. Boo, *DocPrompting expertised by team19*. (2026). [Online]. Available: https://github.com/buuhq-uit/team19-docprompting
[4] P. Yin, B. Deng, E. Chen, B. Vasilescu, and G. Neubig, "Learning to mine aligned code and natural language pairs from stack overflow," in *Proceedings of the 15th International Conference on Mining Software Repositories*, Gothenburg Sweden: ACM, May 2018, pp. 476–486. doi: 10.1145/3196398.3196408.
[5] "DevDocs." [Online]. Available: https://devdocs.io
[6] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple Contrastive Learning of Sentence Embeddings," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 6894–6910. doi: 10.18653/v1/2021.emnlp-main.552.
[7] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, Online: Association for Computational Linguistics, 2021, pp. 874–880. doi: 10.18653/v1/2021.eacl-main.74.